

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

ASAD ARSHADALI GOVE (1BM24CS062)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING**

in

COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

February to June 2026

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by **ASAD ARSHADALI GOVE (1BM24CS062)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Dr. Namratha M
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	6
2	Implement Johnson Trotter algorithm to generate permutations.	8
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	11
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	15
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	17
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	20
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	22
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	24
9	Implement Fractional Knapsack using Greedy technique.	27
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	29
11	Implement "N-Queens Problem" using Backtracking.	33
12	Leetcode Problems	35

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>
#define MAX 10
int main() {
    int n, i, j, graph[MAX][MAX], indegree[MAX] = {0};
    int queue[MAX], front = 0, rear = -1, topo[MAX], k = 0;

    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
            if (graph[i][j] == 1) indegree[j]++;
        }
    }
    for (i = 0; i < n; i++) if (indegree[i] == 0) queue[++rear] = i;
    while (front <= rear) {
        int v = queue[front++]; topo[k++] = v;
        for (j = 0; j < n; j++) {
            if (graph[v][j] == 1) {
                indegree[j]--;
                if (indegree[j] == 0) queue[++rear] = j;
            }
        }
    }
    printf("Topological Order: ");
    for (i = 0; i < k; i++) printf("%d ", topo[i]);
    return 0;
}
```

OUTPUT:

```
Enter number of vertices: 4
Enter number of edges (directed): 4
Enter edges (format: source destination):
0 1
0 2
1 3
2 3
Topological Sequence: 1 2 3 4
```

Lab program 2:

Implement Johnson Trotter algorithm to generate permutations.

```
#include <stdio.h>

int getMobile(int a[], int dir[], int n)
{
    int mobile = 0, mobile_pos = -1;

    for(int i = 0; i < n; i++)
    {
        if(dir[i] == -1 && i != 0 && a[i] > a[i-1]) // Left
        {
            if(a[i] > mobile)
            {
                mobile = a[i];
                mobile_pos = i;
            }
        }

        if(dir[i] == 1 && i != n-1 && a[i] > a[i+1]) // Right
        {
            if(a[i] > mobile)
            {
                mobile = a[i];
                mobile_pos = i;
            }
        }
    }

    return mobile_pos;
}

void printPermutation(int a[], int n)
{
    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

int main()
{
    int n;

    printf("Enter n: ");
```

```

scanf("%d", &n);

int a[n], dir[n];

// Initial permutation
for(int i = 0; i < n; i++)
{
    a[i] = i + 1;
    dir[i] = -1; // -1 = Left, 1 = Right
}

printf("Permutations:\n");
printPermutation(a, n);

while(1)
{
    int pos = getMobile(a, dir, n);

    if(pos == -1)
        break;

    int mobile = a[pos];

    // Swap mobile element
    if(dir[pos] == -1)
    {
        int temp = a[pos];
        a[pos] = a[pos-1];
        a[pos-1] = temp;

        temp = dir[pos];
        dir[pos] = dir[pos-1];
        dir[pos-1] = temp;

        pos--;
    }
    else
    {
        int temp = a[pos];
        a[pos] = a[pos+1];
        a[pos+1] = temp;

        temp = dir[pos];
        dir[pos] = dir[pos+1];
        dir[pos+1] = temp;

        pos++;
    }
}

```

```
}

// Reverse directions of elements larger than mobile
for(int i = 0; i < n; i++)
{
    if(a[i] > mobile)
        dir[i] = -dir[i];
}

printPermutation(a, n);
}

Return
```

OUTPUT:

Enter n: 4

Permutations:

1 2 3 4

1 2 4 3

1 4 2 3

4 1 2 3

4 1 3 2

1 4 3 2

1 3 4 2

1 3 2 4

3 1 2 4

3 1 4 2

3 4 1 2

4 3 1 2

4 3 2 1

3 4 2 1

3 2 4 1

3 2 1 4

2 3 1 4

2 3 4 1

2 4 3 1

4 2 3 1

4 2 1 3

2 4 1 3

2 1 4 3

2 1 3 4

Lab program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

```
#include <stdio.h>

void merge(int a[], int l, int m, int r) {
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;

    int L[n1], R[n2];

    for (i = 0; i < n1; i++) {
        L[i] = a[l + i];
    }
    for (j = 0; j < n2; j++) {
        R[j] = a[m + 1 + j];
    }

    i = 0;
    j = 0;
    k = l;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            a[k] = L[i];
            i++;
        } else {
            a[k] = R[j];
            j++;
        }
    }
}
```

```

    }
    k++;
}

while (i < n1) {
    a[k] = L[i];
    i++;
    k++;
}

while (j < n2) {
    a[k] = R[j];
    j++;
    k++;
}
}

void merge_sort(int a[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        merge_sort(a, l, m);
        merge_sort(a, m + 1, r);
        merge(a, l, m, r);
    }
}

int main() {
    int n;

    printf("Enter the number of elements: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Invalid array size.\n");
        return 1;
    }
}

```

```

}

int arr[n];

printf("Enter %d elements:\n", n);
for (int i = 0; i < n; i++) {
    scanf("%d", &arr[i]);
}

merge_sort(arr, 0, n - 1);

printf("Sorted array: ");
for (int i = 0; i < n; i++) {
    printf("%d ", arr[i]);
}

printf("\n");

```

OUTPUT:

```

Enter number of elements: 50000
Time taken to sort 50000 elements = 0.004000 seconds

```

```

Enter number of elements: 2000
Time taken to sort 2000 elements = 0.001000 seconds

```

```

Enter number of elements: 20000
Time taken to sort 20000 elements = 0.002000 seconds

```

```

Enter number of elements: 1000
Time taken to sort 1000 elements = 0.000000 seconds

```

```

Enter number of elements: 5000
Time taken to sort 5000 elements = 0.001000 seconds

```

Lab program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int a[], int low, int high) {
    int key = a[low];
    int i = low + 1;
    int j = high;

    while (1) {
        while (i <= high && a[i] <= key) {
            i++;
        }

        while (j >= low && a[j] > key) {
            j--;
        }

        if (i >= j) {
            swap(&a[low], &a[j]);
            break;
        }
        swap(&a[i], &a[j]);
    }
    return j;
}

void quicksort(int a[], int low, int high) {
    if (low < high) {
        int j = partition(a, low, high);
        quicksort(a, low, j - 1);
        quicksort(a, j + 1, high);
    }
}
```

```

int main() {
    int n;
    printf("Enter the number of elements: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Invalid array size.\n");
        return 1;
    }

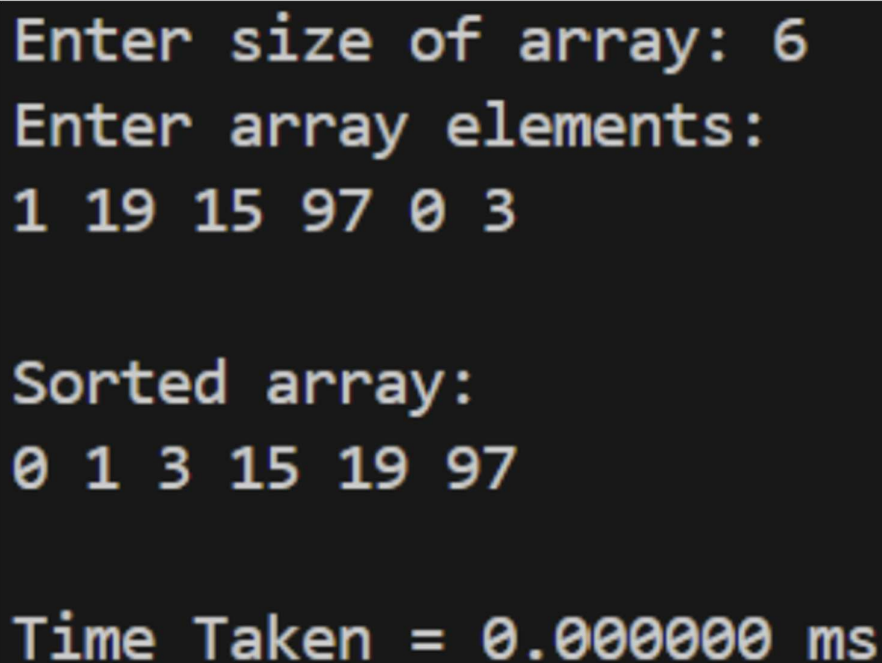
    int arr[n];

    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    quicksort(arr, 0, n - 1);
    printf("Sorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}

```

OUTPUT



```

Enter size of array: 6
Enter array elements:
1 19 15 97 0 3

Sorted array:
0 1 3 15 19 97

Time Taken = 0.000000 ms

```

Lab program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include<stdio.h>
```

```
void heapify(int a[],int n, int p){
    int item = a[p];
    int c = 2*p+1;
    while(c<=n-1){
        if(c+1<=n-1 && a[c+1] > a[c]) c++;
        if(item < a[c]){
            a[p] = a[c];
            p = c;
            c = 2*p+1;
        }
        else break;
    }
    a[p] = item;
}
```

```
void create_heap(int a[], int n){
    for(int i = (n-1)/2; i>=0; i--) heapify(a,n,i);
}
```

```
void heap_sort(int a[],int n){
    create_heap(a,n);
    for(int i = n-1; i>=0; i--){
        int temp = a[0];
        a[0] = a[i];
        a[i] = temp;
        heapify(a,i,0);
    }
}
```

```
void main(){
    int n;
    printf("Enter number of elements: ");
    scanf("%d",&n);
    int a[n];
    printf("Enter the elements of the array :");
```

```
for(int i = 0;i<n;i++){  
    scanf("%d",&a[i]);  
}  
heap_sort(a,n);  
printf("Sorted: ");  
for(int i = 0;i<n;i++){  
    printf("%d ",a[i]);  
}  
}
```

OUTPUT:

```
Enter number of elements: 5  
  
Enter elements:  
45 12 89 3 27  
  
Sorted array:  
3 12 27 45 89  
  
Time taken = 0.005000 seconds
```


Lab program 6:

Implement 0/1 Knapsack problem using dynamic programming.

```
#include <stdio.h>
#include <time.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int knapsack(int W, int wt[], int val[], int n) {
    int K[n+1][W+1];

    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {

            if (i == 0 || w == 0)
                K[i][w] = 0;

            else if (wt[i-1] <= w)
                K[i][w] = max(val[i-1] + K[i-1][w - wt[i-1]], K[i-1][w]);

            else
                K[i][w] = K[i-1][w];
        }
    }

    return K[n][W];
}

int main() {
    int n, W;

    printf("Enter number of items: ");
    scanf("%d", &n);

    int wt[n], val[n];

    printf("Enter weights:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &wt[i]);

    printf("Enter values:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &val[i]);

    printf("Enter knapsack capacity: ");
    scanf("%d", &W);

    clock_t start, end;
    start = clock();

    int result = knapsack(W, wt, val, n);
```

```
end = clock();  
double time_taken = (((double)(end - start)) / CLOCKS_PER_SEC)*1000;  
printf("Maximum Profit = %d\n", result);  
printf("Execution Time = %f\n", time_taken);  
  
return 0;  
}
```

OUTPUT:

```
Enter number of items: 4  
Enter weights:  
2 3 4 5  
Enter values:  
3 4 5 6  
Enter knapsack capacity: 5  
Maximum Profit = 7  
Execution Time = 0.000000 seconds
```

Lab program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include<stdio.h>
void Floyd(int n, int
    cost[50][50], int
    A[50][50])
{
    int i, j, k;

    for(i = 1; i <= n; i++)
        for(j = 1; j <= n; j++)
            A[i][j] = cost[i][j];

    for(k = 1; k <= n; k++)
        for(i = 1; i <= n; i++)
            for(j = 1; j <= n;
                j++)
                if(A[i][k] +
                    A[k][j] < A[i][j])
                    A[i][j] =
                        A[i][k] + A[k][j];
}

int main() {
    int n, i, j;
    int cost[50][50],
    A[50][50];
    printf("Enter number of
    vertices: ");
    scanf("%d", &n);
    printf("Enter the cost
    matrix (use 999 for
    infinity):\n");
    for(i = 1; i <= n; i++) {
        for(j = 1; j <= n; j++) {
            scanf("%d",
                &cost[i][j]);
            if (cost[i][j] == 0
                && i != j) {
                cost[i][j] = 999;
            }
        }
    }
    Floyd(n, cost, A);
    printf("\nThe all-pairs
    shortest path matrix
    is:\n");
    for(i = 1; i <= n; i++) {
        for(j = 1; j <= n; j++) {
            if (A[i][j] == 999)
                printf("INF ");
            else printf("%d ",
                A[i][j]);
        }
    }
}
```

```
        printf("\n");  
    }  
  
    return 0;  
}
```

OUTPUT:

```
Enter number of vertices: 4  
Enter adjacency matrix:  
(Enter 99 for no edge)  
0 5 99 10  
99 0 3 99  
99 99 0 1  
99 99 99 0  
  
All Pair Shortest Path Matrix:  
  0   5   8   9  
INF  0   3   4  
INF INF  0   1  
INF INF INF  0
```

Lab program 8(a):

Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
#include <stdio.h>

int main()
{
    int n, cost[10][10];
    int visited[10] = {0};
    int min, u, v, edges = 0;
    int mincost = 0;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter cost matrix:\n");
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);

            if(cost[i][j] == 0)
                cost[i][j] = 999; // No edge
        }
    }

    visited[0] = 1; // Start from vertex 0

    printf("\nEdges in MST:\n");

    while(edges < n - 1)
    {
        min = 999;

        for(int i = 0; i < n; i++)
        {
            if(visited[i])
            {
                for(int j = 0; j < n; j++)
                {
                    if(!visited[j] && cost[i][j] < min)
                    {
                        min = cost[i][j];
                        u = i;
                        v = j;
                    }
                }
            }
        }

        printf("%d -> %d : %d\n", u, v, min);

        mincost += min;
        visited[v] = 1;
        edges++;
    }
}
```

```
printf("\nTotal minimum Cost : %d\n", mincost);  
return 0;  
}
```

OUTPUT:

```
Enter the adjacency matrix:  
0 2 0 6 0  
2 0 3 8 5  
0 3 0 0 7  
6 8 0 0 9  
0 5 7 9 0  
  
Edge      Weight  
0 - 1     2  
1 - 2     3  
0 - 3     6  
1 - 4     5  
Total cost of MST = 16
```

Lab program 8(b):

Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
#include <stdio.h>
```

```
void Floyd's(int n, int cost[50][50], int A[50][50])
```

```
{
    int i, j, k;

    for(i = 1; i <= n; i++)
        for(j = 1; j <= n; j++)
            A[i][j] = cost[i][j];

    for(k = 1; k <= n; k++)
        for(i = 1; i <= n; i++)
            for(j = 1; j <= n; j++)
                if(A[i][k] + A[k][j] < A[i][j])
                    A[i][j] = A[i][k] + A[k][j];
}
```

```
int main() {
    int n, i, j;
    int cost[50][50], A[50][50];
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter the cost matrix (use 999 for infinity):\n");
    for(i = 1; i <= n; i++) {
        for(j = 1; j <= n; j++) {
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0 && i != j) {
                cost[i][j] = 999;
            }
        }
    }
    Floyd's(n, cost, A);
    printf("\nThe all-pairs shortest path matrix is:\n");
    for(i = 1; i <= n; i++) {
        for(j = 1; j <= n; j++) {
            if (A[i][j] == 999) printf("INF ");
            else printf("%d ", A[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

OUTPUT:

```
Enter number of items: 4
Enter weights:
2 3 4 5
Enter values:
3 4 5 6
Enter knapsack capacity: 5
Maximum Profit = 7
```


LAB PROGRAM 9

Implement Fractional Knapsack using Greedy technique.

```
#include<stdio.h>
struct item{
    int p;
    int w;
    float r;
    float x;
};
int main(){
    int n, m;
    printf("Enter the number of items: ");
    scanf("%d", &n);
    printf("Enter the maximum capacity: ");
    scanf("%d", &m);
    struct item items[20], temp;
    for(int i=0; i<n; i++){
        printf("Enter weight of item %d: ", i+1);
        scanf("%d", &items[i].w);
        printf("Enter profit of item %d: ", i+1);
        scanf("%d", &items[i].p);
        items[i].x = 0;
        items[i].r = (float)items[i].p / items[i].w;
    }
    for(int i=0; i<n; i++){
        for(int j=i+1; j<n; j++){
            if(items[j].r > items[i].r){
                temp = items[i];
                items[i] = items[j];
                items[j] = temp;
            }
        }
    }
    float remain = m, maxprofit = 0;
    for(int i=0; i<n; i++){
        if(items[i].w <= remain){
            items[i].x = 1;
            remain -= items[i].w;
            maxprofit += items[i].p;
        } else {
            items[i].x = remain / items[i].w;
            maxprofit += items[i].p * items[i].x;
            break;
        }
    }

    printf("\nMaximum profit = %.2f\n", maxprofit);
}
```

OUTPUT:

```
Enter number of objects: 3
Enter max capacity of Knapsack: 20

Enter profit, weight for 3 objects
25 18
24 15
15 10

Maximum profit: 31.500000
```

Lab program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include <stdio.h>

int main()
{
    int n, cost[10][10];
    int dist[10], visited[10];
    int source;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter cost matrix:\n");
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);

            if(i != j && cost[i][j] == 0)
                cost[i][j] = 999;
        }
    }

    printf("Enter source vertex: ");
    scanf("%d", &source);

    for(int i = 0; i < n; i++)
    {
        dist[i] = cost[source][i];
        visited[i] = 0;
    }

    dist[source] = 0;
    visited[source] = 1;

    for(int count = 1; count < n; count++)
    {
        int min = 999, u = -1;

        // Find nearest unvisited vertex
        for(int i = 0; i < n; i++)
        {
            if(!visited[i] && dist[i] < min)
            {
                min = dist[i];
                u = i;
            }
        }
    }
}
```

```

    visited[u] = 1;

    // Relax adjacent vertices
    for(int v = 0; v < n; v++)
    {
        if(!visited[v] &&
            dist[u] + cost[u][v] < dist[v])
        {
            dist[v] = dist[u] + cost[u][v];
        }
    }
}

printf("\nShortest distances from vertex %d:\n", source);

for(int i = 0; i < n; i++)
{
    printf("%d -> %d = %d\n", source, i, dist[i]);
}

return 0;
}

```

OUTPUT:

```

Enter the number of vertices: 3
Enter the cost matrix:
0 4 99999
4 0 99999
99999 99999 0
Enter the source vertex: 1

Shortest distances from vertex 1:
To vertex 0 = 4
To vertex 1 = 0
To vertex 2 = 99999

```

Lab program 11:

Implement “N-Queens Problem” using Backtracking.

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 20
int x[MAX];
int PLACE(int k, int i)
{
    int j;
    for (j = 1; j <= k - 1; j++)
    {
        if ((x[j] == i) || (abs(x[j] - i) == abs(j - k)))
            return 0;
    }
    return 1;
}

void printSolution(int n)
{
    int i, j;
    printf("\nSolution:\n");
    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= n; j++)
        {
            if (x[i] == j)
                printf(" Q ");
            else
                printf(" . ");
        }
        printf("\n");
    }
}

void NQueens(int n)
{
    int k = 1;
    x[k] = 0;
    while (k != 0)
    {
        x[k] = x[k] + 1;
        while ((x[k] <= n) && !PLACE(k, x[k]))
        {
            x[k] = x[k] + 1;
        }
        if (x[k] <= n)
        {
            if (k == n)
            {
                printSolution(n);
            }
            else
            {
                k++;
                x[k] = 0;
            }
        }
    }
}
```

```

        else
        {
            k--;
        }
    }
}

int main()
{
    int n;
    printf("Enter number of queens: ");
    scanf("%d", &n);
    NQueens(n);
}

```

OUTPUT:

```

Enter number of queens: 4

Solution:
. Q . .
. . . Q
Q . . .
. . Q .

Solution:
. . Q .
Q . . .
. . . Q
. Q . .

```

LEETCODE

The image displays three screenshots of a LeetCode submission interface, showing the code, runtime, and memory usage for different problems.

Top Screenshot: The interface shows the "Code" tab with a C++ solution. The runtime is 0 ms, Beats 100.00%, and memory is 12.11 MB, Beats 93.79%. The code defines a `swap` function and a `backtrack` function. The runtime graph shows a single bar at 0 ms, indicating that 4.81% of solutions used 3 ms of runtime.

Middle Screenshot: The interface shows the "Code" tab with a C++ solution. The runtime is 0 ms, Beats 100.00%, and memory is 10.58 MB, Beats 34.97%. The code defines a `findFirst` function. The runtime graph shows a single bar at 0 ms.

Bottom Screenshot: The interface shows the "Code" tab with a C++ solution. The runtime is 2 ms, Beats 55.56%, and memory is 16.39 MB, Beats 74.07%. The code defines a `minSwap` function. The runtime graph shows a bar at 2 ms, indicating that 55.56% of solutions used 2 ms of runtime.

